ĐẠI HỌC PHENIKAA

TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA



BÁO CÁO TIẾN ĐỘ ĐỒ ÁN

HỌC PHẦN ỨNG DỤNG PHÂN TÁN

ĐỀ TÀI CHATAPP

XÂY DỰNG ỨNG DỤNG CHAT NHÓM THEO KIẾN TRÚC MICROSERVICES

Giai đoạn hình thành ý tưởng và thiết kế kiến trúc

Giảng viên hướng dẫn: .....

Sinh viên thực hiện: Nguyễn Trọng Dương

Nguyễn Hữu Huy

Khóa: K17 – 2023–2027

Lớp tín chỉ: Ứng dụng phân tán - 4-1-26(NO1)

Chương trình đào tạo: Đại học chính quy

Hà Nội, tháng 10 năm 2026

MỤC LỤC

CHƯƠNG 1 MỞ ĐẦU 1

1.1 Lý do lựa chọn đề tài 1

1.2 Mục tiêu của đề tài 1

1.3 Yêu cầu và phạm vi nghiên cứu 1

1.4 Phương pháp thực hiện 1

CHƯƠNG 2 KIẾN TRÚC HỆ THỐNG 3

2.1 Lựa chọn kiến trúc 3

2.2 Sơ đồ kiến trúc tổng thể 3

2.3 Phân chia dịch vụ và trách nhiệm 4

2.4 Tổ chức dữ liệu và hạ tầng 4

2.5 Cách thức giao tiếp và phối hợp 5

2.6 Kết quả đạt được ở giai đoạn báo cáo 5

2.7 Kế hoạch tiếp theo 5

CHƯƠNG 1 MỞ ĐẦU

1.1 Lý do lựa chọn đề tài

Trao đổi tin nhắn theo nhóm là nhu cầu thường gặp trong học tập và làm việc. Một ứng dụng chat cần tiếp nhận nhiều kết nối cùng lúc, chuyển tin đến người nhận và lưu lịch sử hội thoại. Các chức năng tài khoản, nhóm, tin nhắn và thông báo có nhiệm vụ khác nhau nhưng phải phối hợp để tạo thành một hệ thống thống nhất.

Từ nhu cầu đó, nhóm lựa chọn đề tài ChatApp để vận dụng kiến thức của học phần Ứng dụng phân tán. Đề tài cho phép làm rõ cách chia một ứng dụng thành các dịch vụ độc lập, giao tiếp qua mạng và quản lý dữ liệu tại nhiều thành phần. Trong giai đoạn đầu, nhóm tập trung xác định yêu cầu và xây dựng kiến trúc làm cơ sở triển khai.

1.2 Mục tiêu của đề tài

Mục tiêu nghiệp vụ là xây dựng ứng dụng chat nhóm trên nền web, cho phép người dùng đăng ký, đăng nhập, tạo nhóm, quản lý thành viên, gửi và nhận tin nhắn theo thời gian thực, xem lịch sử và theo dõi số tin chưa đọc.

Mục tiêu kỹ thuật là thiết kế hệ thống theo kiến trúc microservices kết hợp hướng sự kiện; xác định ranh giới trách nhiệm và quyền sở hữu dữ liệu của từng dịch vụ; lựa chọn cách giao tiếp phù hợp giữa giao diện, dịch vụ và hạ tầng. Kiến trúc cần tạo điều kiện triển khai từng dịch vụ và mở rộng thành phần có nhiều kết nối.

1.3 Yêu cầu và phạm vi nghiên cứu

Về chức năng, hệ thống gồm bốn nhóm yêu cầu: quản lý tài khoản; quản lý nhóm và thành viên; xử lý tin nhắn và lịch sử; quản lý thông báo tin chưa đọc. Người dùng cần đăng nhập để truy cập chức năng và chỉ được thao tác với nhóm theo quyền của mình.

Về yêu cầu phi chức năng, hệ thống cần hỗ trợ nhiều người dùng đồng thời, có khả năng phát triển từng dịch vụ riêng và hạn chế phụ thuộc dữ liệu giữa các dịch vụ. Trong môi trường học tập, các thành phần có thể chạy trên cùng máy bằng những tiến trình hoặc container khác nhau; việc giao tiếp vẫn thông qua giao diện mạng.

Phạm vi báo cáo này gồm hình thành ý tưởng, phân tích yêu cầu và thiết kế kiến trúc đến hết Chương 2. Các cơ chế đồng bộ hóa, sao lưu và chịu lỗi sẽ được nghiên cứu ở những giai đoạn sau.

1.4 Phương pháp thực hiện

Nhóm thực hiện theo hướng: xác định chức năng cốt lõi, phân chia miền nghiệp vụ, lập sơ đồ kiến trúc, xác định dữ liệu và hợp đồng giao tiếp, sau đó xây dựng khung dự

án. Nội dung được đối chiếu với bảng tiêu chí của học phần [1] và tài liệu thiết kế ChatApp [2].

CHƯƠNG 2 KIẾN TRÚC HỆ THỐNG

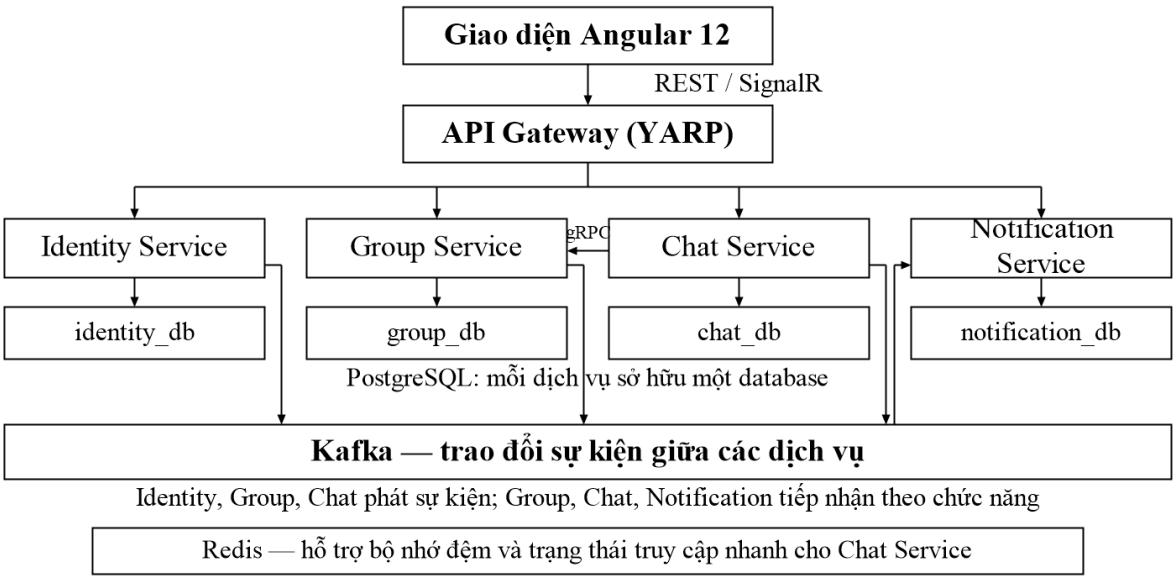
2.1 Lựa chọn kiến trúc

Nhóm lựa chọn kiến trúc microservices để tách các chức năng tài khoản, nhóm, tin nhắn và thông báo thành bốn dịch vụ nghiệp vụ. Mỗi dịch vụ có giao diện truy cập và cơ sở dữ liệu riêng. API Gateway là điểm tiếp nhận yêu cầu từ giao diện, giúp người dùng truy cập hệ thống thông qua một địa chỉ thống nhất.

Kiến trúc hướng sự kiện được kết hợp để các dịch vụ trao đổi những thay đổi không cần phản hồi ngay. Dịch vụ phát sự kiện lên Kafka; dịch vụ quan tâm tiếp nhận và cập nhật dữ liệu của mình. Cách tổ chức này phù hợp với việc cập nhật thông tin người dùng và thông báo tin chưa đọc mà không buộc mọi thao tác phải gọi nối tiếp nhau.

So với việc đặt toàn bộ chức năng trong một ứng dụng, cách chia này làm rõ trách nhiệm và thuận lợi cho triển khai từng phần. Đổi lại, hệ thống phải quản lý nhiều tiến trình, xử lý giao tiếp qua mạng và chấp nhận độ trễ cập nhật giữa các bản sao dữ liệu. Vì vậy, nhóm giữ phạm vi ở bốn dịch vụ nghiệp vụ để phù hợp quy mô đồ án.

2.2 Sơ đồ kiến trúc tổng thể



Hình 2.1 Kiến trúc dự kiến của hệ thống ChatApp

Giao diện Angular kết nối đến API Gateway; Gateway chuyển yêu cầu đến dịch vụ phù hợp. Chat Service trao đổi trực tiếp với Group Service qua gRPC khi cần kiểm tra thành viên. Kafka phục vụ trao đổi sự kiện; PostgreSQL lưu dữ liệu nghiệp vụ và Redis hỗ trợ dữ liệu truy cập nhanh. Sơ đồ thể hiện thiết kế tổng thể, không đồng nghĩa mọi thành phần đã hoàn thiện tại mốc báo cáo.

2.3 Phân chia dịch vụ và trách nhiệm

Ranh giới dịch vụ được xác định theo nghiệp vụ và dữ liệu cần quản lý, thay vì chia theo từng màn hình. Bốn dịch vụ cùng API Gateway được phân công như sau:

Thành phần	Trách nhiệm và dữ liệu quản lý
Identity Service	Đăng ký, đăng nhập và cung cấp thông tin tài khoản. Sở hữu identity_db; là nguồn dữ liệu gốc về người dùng.
Group Service	Tạo và quản lý nhóm, quản lý thành viên và vai trò. Sở hữu group_db; cung cấp giao diện kiểm tra thành viên cho Chat Service.
Chat Service	Tiếp nhận và chuyển tin nhắn theo thời gian thực, lưu và truy vấn lịch sử. Sở hữu chat_db; sử dụng SignalR cho kết nối với người dùng.
Notification Service	Tiếp nhận sự kiện tin nhắn và thay đổi thành viên; quản lý số tin chưa đọc. Sở hữu notification_db.
API Gateway	Dùng YARP để định tuyến yêu cầu từ giao diện và kiểm tra truy cập tại cửa vào. Không sở hữu database nghiệp vụ.

Bảng 2.1 Phân chia trách nhiệm giữa các thành phần

2.4 Tổ chức dữ liệu và hạ tầng

Hệ thống áp dụng nguyên tắc database per service: mỗi dịch vụ chỉ trực tiếp đọc và ghi database do mình sở hữu. Không thiết lập khóa ngoại giữa database của các dịch vụ. Khi cần dữ liệu từ dịch vụ khác, thành phần sử dụng API hoặc tiếp nhận sự kiện, tránh truy cập chéo database.

Trong môi trường đồ án, bốn database được bố trí trên cùng một máy chủ PostgreSQL để tiết kiệm tài nguyên. Đây là phân tách dữ liệu ở mức logic; không phải bốn máy chủ cơ sở dữ liệu độc lập. Ví dụ, Group Service có thể lưu bản sao tên hiển thị của người dùng từ sự kiện do Identity Service phát ra, nhưng quyền quản lý tài khoản vẫn thuộc Identity Service.

Backend sử dụng ASP.NET Core Web API (.NET 10), truy cập PostgreSQL bằng EF Core. Frontend sử dụng Angular 12 tương thích Node 14. Kafka là hạ tầng sự kiện; Redis hỗ trợ bộ nhớ đệm và trạng thái truy cập nhanh. Docker Compose tổ chức môi trường phát triển. Các công nghệ này là thành phần hỗ trợ, không được tính là dịch vụ nghiệp vụ.

2.5 Cách thức giao tiếp và phối hợp

Giao tiếp được chia theo mục đích: REST dùng cho yêu cầu từ giao diện như đăng nhập, tạo nhóm và xem lịch sử; SignalR dùng để gửi và nhận tin theo thời gian thực qua Gateway. Khi cần kết quả ngay từ dịch vụ khác, Chat Service gọi gRPC đến Group Service để kiểm tra quyền thành viên. Các lời gọi nội bộ này không đi vòng qua Gateway.

Với thay đổi không cần phản hồi tức thời, các dịch vụ trao đổi sự kiện qua Kafka. Ví dụ, Identity Service phát sự kiện người dùng đăng ký để Group Service cập nhật bản sao thông tin; Chat Service phát sự kiện tin nhắn mới để Notification Service cập nhật số chưa đọc. Các bản sao áp dụng tính nhất quán cuối cùng: sau khi dữ liệu gốc thay đổi, dịch vụ nhận có thể cần một khoảng thời gian để cập nhật.

Luồng nghiệp vụ dự kiến khi gửi tin: người dùng mở nhóm và gửi nội dung qua SignalR; Chat Service kiểm tra thành viên, lưu tin vào chat_db rồi chuyển tin đến các kết nối trong nhóm. Sau đó, sự kiện tin nhắn được chuyển qua Kafka để Notification Service xử lý số chưa đọc. Việc tách xử lý thông báo giúp chức năng chat không phải chờ cập nhật thông báo hoàn tất.

2.6 Kết quả đạt được ở giai đoạn báo cáo

Đến mốc này, nhóm đã xác định đề tài và các chức năng cốt lõi; lựa chọn kiến trúc microservices kết hợp hướng sự kiện; hoàn thành sơ đồ tổng thể, phân chia trách nhiệm và xác định quyền sở hữu dữ liệu của bốn dịch vụ. Nhóm cũng đã xác định vai trò của Gateway và các hình thức giao tiếp chính.

Về nền tảng triển khai, dự án đã có cấu trúc solution gồm Gateway, bốn dịch vụ và hai thư viện dùng chung: Contracts chứa hợp đồng giao tiếp, Common chứa tiện ích hạ tầng. Các dịch vụ không tham chiếu trực tiếp mã nguồn của nhau. Cấu hình môi trường hạ tầng và khởi tạo bốn database đã được xây dựng. Những kết quả này tạo cơ sở để tiếp tục triển khai theo từng chương [2], [3].

2.7 Kế hoạch tiếp theo

Ở Chương 3, nhóm sẽ tập trung mô tả mô hình tiến trình và luồng: cách các dịch vụ chạy độc lập, xử lý nhiều yêu cầu bằng async/await và tổ chức tác vụ nền tiếp nhận sự kiện. Nhóm sẽ xây dựng sơ đồ xử lý, ghi nhận cách khởi động các thành phần và bổ sung minh chứng cho phần tiến trình, luồng.

TÀI LIỆU THAM KHẢO

[1] Bảng chấm điểm hệ thống phân tán, tài liệu tiêu chí đánh giá học phần do giảng viên cung cấp.

[2] Nhóm thực hiện ChatApp, Thiết kế hệ thống ChatApp, tài liệu nội bộ docs/DESIGN.md.

[3] Nhóm thực hiện ChatApp, Kế hoạch triển khai và ghi chú tiến độ, tài liệu nội bộ docs/PLAN.md và docs/PROGRESS.md.